



Università degli Studi di Salerno

Dipartimento di Ingegneria dell'Informazione ed Elettrica
e Matematica Applicata (DIEM)

TripMate

Applicazione mobile per la pianificazione di viaggi, tappe, checklist e spese, funzionante offline

Relazione di Progetto

Corso di Mobile Programming
A.A. 2025-2026

Componente	Matricola
Armando Incolingo	0612708776
Maresca Luca	0612709984
Simona Spagnuolo	0612709947
Deborah Tramontano	0612709139
Matteo Trivellone	0612709465

Numero Gruppo: 11

Luglio 2026

Indice

1	Descrizione dell'app	2
1.1	Obiettivo dell'applicazione	2
1.2	Tipologia di utenti	2
1.3	Problema che l'app intende risolvere	2
1.4	Principali scenari d'uso	2
2	Requisiti	3
2.1	Requisiti funzionali implementati	3
2.2	Requisiti non funzionali	3
2.3	Funzionalità aggiuntive	4
3	Progettazione dell'app	4
3.1	Struttura generale dell'app	4
3.2	Flusso di navigazione	4
3.3	Schermate principali	5
3.3.1	Elenco viaggi	5
3.3.2	Dettaglio viaggio	5
3.3.3	Timeline	6
3.3.4	Statistiche	6
3.4	Organizzazione dei dati	6
4	Scelte tecnologiche	7
4.1	Framework utilizzato	7
4.2	Librerie principali	8
4.3	Modalità di gestione della persistenza	8
5	Implementazione	8
5.1	Organizzazione del codice	8
5.2	Modello dei dati	9
5.3	Gestione dello stato	9
5.4	Gestione della navigazione	10
5.5	Gestione dei dati persistenti e integrità referenziale	10
5.6	Componenti e widget principali	11
5.7	Approfondimenti implementativi	11
5.7.1	Calcolo automatico dello stato del viaggio	11
5.7.2	Generazione automatica delle giornate	12
5.7.3	Duplicazione di un viaggio	12
5.7.4	Packing list generata dal tipo di viaggio	13
5.8	Architettura a livelli in sintesi	13
6	Schermate dell'applicazione	14
7	Conclusioni	15

1 Descrizione dell'app

TripMate è un'applicazione mobile che accompagna l'utente in tutte le fasi di un viaggio: dalla pianificazione iniziale fino al riepilogo delle spese. L'app consente di gestire uno o più viaggi, suddividerli in giornate, associare attività a ciascuna giornata, preparare checklist di ciò che occorre portare o fare, annotare informazioni utili (prenotazioni, indirizzi, promemoria) e tenere sotto controllo il budget. Tutti i dati risiedono sul dispositivo e restano consultabili anche in assenza di connessione di rete, requisito centrale del dominio applicativo: chi viaggia si trova spesso senza rete, in roaming o in aereo.

1.1 Obiettivo dell'applicazione

L'obiettivo dell'applicazione è centralizzare in un unico strumento le informazioni che compongono l'organizzazione di un viaggio. A questo scopo l'applicazione definisce un modello dei dati unico e mette a disposizione le funzioni necessarie a gestirlo, con l'intento di rendere tali informazioni complete, consultabili e disponibili sul dispositivo anche in assenza di rete. In particolare, l'applicazione consente di:

- gestire in modo strutturato le entità che descrivono un viaggio — viaggi, giornate, attività, checklist, spese e note — tramite operazioni complete di creazione, consultazione, modifica ed eliminazione;
- mantenere una vista aggregata e costantemente aggiornata dello stato di avanzamento, quali le attività completate, gli elementi della checklist ancora da spuntare e il budget residuo;
- automatizzare le operazioni ripetitive, ossia la generazione delle giornate a partire dalle date del viaggio, la produzione di una checklist di preparazione in base alla tipologia e la duplicazione di un viaggio esistente;
- garantire il funzionamento in assenza di connettività e la persistenza dei dati tra sessioni successive, per mezzo di un'archiviazione locale sul dispositivo.

1.2 Tipologia di utenti

L'applicazione si rivolge a chiunque voglia organizzare in autonomia i propri viaggi, dal viaggiatore occasionale a quello abituale. L'uso non richiede competenze tecniche: tutte le funzioni sono accessibili tramite liste, schede e moduli, con un'interfaccia in italiano.

1.3 Problema che l'app intende risolvere

Le informazioni di un viaggio tendono a restare sparse su strumenti diversi e scollegati tra loro. Questa dispersione produce due problemi ricorrenti: la difficoltà di avere una visione unitaria del viaggio e l'indisponibilità dei dati proprio quando servono, cioè in movimento e spesso offline. TripMate affronta entrambi gli aspetti unificando i dati in un modello coerente e salvandoli localmente, così che siano sempre a portata di mano.

1.4 Principali scenari d'uso

Scenario A — Pianificazione. L'utente crea un nuovo viaggio indicando titolo, destinazione, date, budget e tipologia. L'app genera automaticamente una giornata per ogni data del soggiorno; l'utente le arricchisce con località e attività (visite, pasti, spostamenti), assegnando orario, categoria e costo previsto a ciascuna.

Scenario B — Preparazione. Dalla scheda del viaggio l'utente genera una packing list coerente con la tipologia (es. mare, montagna, città), la personalizza spuntando o aggiungendo voci, e registra le prenotazioni come note classificate per tipo.

Scenario C — Durante e dopo il viaggio. L'utente consulta la timeline giornaliera, marca le attività come svolte o annullate, registra le spese sostenute distinguendole da quelle solo previste e monitora il budget residuo. La sezione Statistiche restituisce una lettura

aggregata su tutti i viaggi.

Scenario D — Riuso. Per un viaggio simile a uno già fatto, l'utente duplica il viaggio esistente ottenendo una copia con itinerario, checklist e spese pronti, con gli stati azzerati.

2 Requisiti

2.1 Requisiti funzionali implementati

La tabella seguente riassume i requisiti funzionali (RF) realizzati, con la schermata o il componente in cui sono resi disponibili. Tutte le entità principali supportano il ciclo completo di operazioni CRUD (inserimento, visualizzazione, modifica, cancellazione).

ID	Descrizione	Schermata
RF-01	Elenco dei viaggi con ricerca testuale e filtro per stato	Elenco viaggi
RF-02	Creazione, modifica ed eliminazione di un viaggio	Elenco / Dettaglio
RF-03	Consultazione del dettaglio di un viaggio (banner + schede)	Dettaglio viaggio
RF-04	Suddivisione del viaggio in giornate, con generazione automatica dalle date	Scheda Itinerario
RF-05	Gestione delle attività per giornata (orario, luogo, categoria, costo, stato)	Scheda Itinerario
RF-06	Cambio di stato dell'attività (da fare / fatta / annullata)	Scheda Itinerario
RF-07	Gestione di checklist raggruppate, con spunta e progresso	Scheda Checklist
RF-08	Gestione di note e informazioni utili classificate per tipo	Scheda Info
RF-09	Gestione delle spese con distinzione previste / effettive	Scheda Spese
RF-10	Confronto tra budget e spesa, con residuo/sforamento e ripartizione per categoria	Scheda Spese
RF-11	Filtri (stato viaggio, categoria spesa, stato voci checklist)	Varie schermate
RF-12	Riepiloghi e statistiche aggregate su tutti i viaggi	Statistiche
RF-13	Timeline giornaliera dell'itinerario	Timeline
RF-14	Duplicazione completa di un viaggio	Dettaglio viaggio
RF-15	Generazione di packing list dal tipo di viaggio	Scheda Checklist

2.2 Requisiti non funzionali

ID	Descrizione
RNF-01	Funzionamento offline: nessuna dipendenza da API remote o backend; tutte le funzioni operano su dati locali.
RNF-02	Persistenza: i dati sopravvivono alla chiusura dell'app tramite archiviazione locale su dispositivo.
RNF-03	Coerenza dei dati: l'eliminazione di un viaggio rimuove a cascata tutte le entità collegate, evitando record orfani.
RNF-04	Manutenibilità: codice modulare organizzato per responsabilità (tipi, store, schermate, form, componenti, utilità) e tipizzato staticamente con TypeScript in modalità strict.
RNF-05	Reattività dell'interfaccia: aggiornamento immediato della UI ad ogni modifica dello stato, grazie a uno store reattivo.
RNF-06	Usabilità: linguaggio in italiano, palette coerente, azioni distruttive protette da conferma, moduli con validazione minima.
RNF-07	Portabilità: singola codebase eseguibile su iOS e Android grazie a React Native/-Expo.

2.3 Funzionalità aggiuntive

Oltre alla gestione di base, l'applicazione integra **tre** funzionalità aggiuntive che ne ampliano il valore d'uso; ciascuna è inserita organicamente nel flusso dell'applicazione e non semplicemente accostata.

1. **Timeline giornaliera dell'itinerario.** Una vista verticale, in stile linea del tempo, che dispone in sequenza le giornate del viaggio e, sotto ciascuna, le relative attività ordinate per orario. Offre una lettura d'insieme dell'intero itinerario complementare alla vista giorno-per-giorno.
2. **Duplicazione di un viaggio.** Un servizio dedicato clona un viaggio con l'intero grafo delle entità collegate (giornate, attività, checklist, spese, note), rigenerando gli identificatori e rimappando correttamente i riferimenti, e azzerando gli stati (attività "da fare", voci non spuntate, spese "previste").
3. **Packing list generata dal tipo di viaggio.** A partire da template per categoria (mare, montagna, città, roadtrip, ecc.) combinati con un blocco base comune, l'app genera automaticamente le voci di checklist mancanti, evitando i duplicati già presenti.

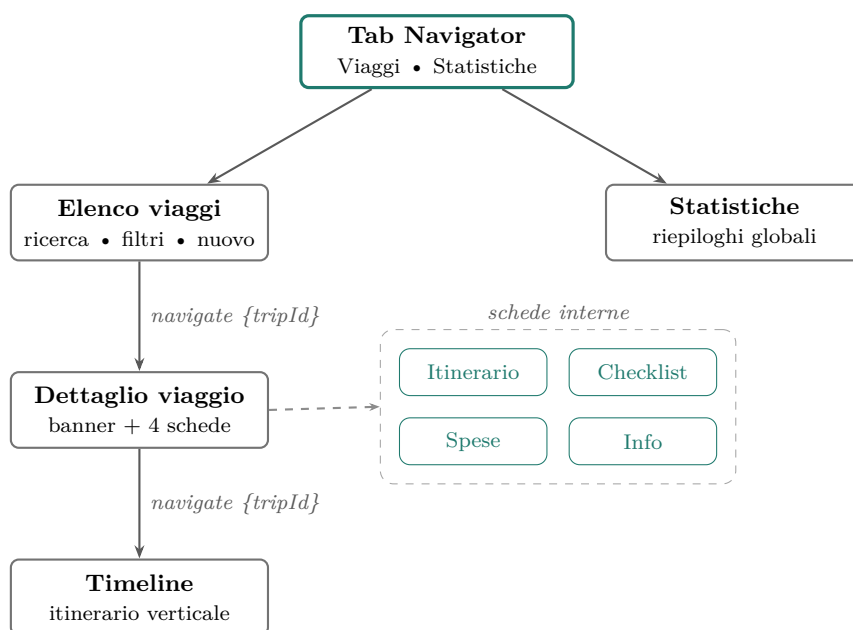
3 Progettazione dell'app

3.1 Struttura generale dell'app

L'app è organizzata attorno a una **barra di navigazione inferiore** con due sezioni: Viaggi e Statistiche. La sezione Viaggi è a sua volta uno stack di schermate, che permette di scendere in profondità dall'elenco al dettaglio del singolo viaggio fino alla timeline, con la possibilità di tornare indietro in qualsiasi momento tramite il gesto di ritorno del sistema operativo. All'interno della schermata di dettaglio, il contenuto è ulteriormente suddiviso in **quattro schede interne** (Itinerario, Checklist, Spese, Info) selezionabili senza lasciare la schermata corrente. In questo modo l'applicazione riesce a offrire molte funzioni mantenendo comunque una struttura chiara e con pochi livelli di profondità.

3.2 Flusso di navigazione

Il diagramma seguente mostra le rotte dell'applicazione e il modo in cui i dati (in particolare l'identificatore del viaggio, `tripId`) vengono passati da una schermata all'altra.



Il passaggio di dati tra schermate avviene trasmettendo alla schermata di destinazione l'identificatore del viaggio: l'elenco apre il dettaglio passando il `tripId`, e il dettaglio apre la timeline propagando lo stesso identificatore. Le schede interne al dettaglio, invece, non sono schermate separate: sono selezionate da uno stato locale e ricevono il `tripId` come proprietà, così da evitare navigazioni superflue.

3.3 Schermate principali

3.3.1 Elenco viaggi

È la schermata iniziale della sezione Viaggi e la porta d'accesso all'intera applicazione. In alto presenta un campo di **ricerca** testuale, che filtra in tempo reale sui campi titolo e destinazione, e una fila di **pulsanti di filtro** per stato (Tutti, In programma, In corso, Completati). I due criteri agiscono insieme: la lista mostra solo i viaggi che corrispondono contemporaneamente al filtro di stato selezionato e al testo cercato.

Sotto, una lista scorrevole (`FlatList`) presenta i viaggi come card (`TripCard`): copertina colorata ricavata dal colore associato al viaggio, destinazione, titolo, intervallo di date e badge di stato. Lo stato mostrato non è un dato inserito manualmente ma è calcolato dalle date rispetto alla data odierna (Sezione 5.2), quindi sempre coerente senza intervento dell'utente. Sul piano dell'interazione, il tocco sulla card apre il dettaglio del viaggio, mentre la pressione prolungata apre direttamente il modulo di modifica: assegnare l'azione più frequente (consultare) al semplice tocco e quella meno frequente (modificare) alla pressione prolungata mantiene la lista pulita, senza aggiungere comandi su ogni scheda. Un pulsante flottante (FAB) in basso a destra avvia la creazione di un nuovo viaggio tramite lo stesso modulo, riutilizzato in modalità inserimento; quando la lista è vuota, un messaggio dedicato guida l'utente al primo inserimento.

3.3.2 Dettaglio viaggio

È il centro operativo dell'applicazione. In testa mostra un **banner** colorato con la sfumatura ricavata dai colori del viaggio, su cui compaiono lo stato, la destinazione e il titolo, insieme ai comandi per tornare indietro, aprire la Timeline e Duplicare il viaggio. Al di sotto, il contenuto è ripartito in **quattro schede** che si alternano restando nella stessa schermata, così che il passaggio da una all'altra sia immediato:

- **Itinerario** — una fila orizzontale e scorrevole di pulsanti, uno per giornata, consente di scegliere il giorno da consultare; alla prima apertura le giornate vengono generate automaticamente dalle date del viaggio (Sezione 5.2). Per il giorno selezionato sono mostrati l'intestazione (titolo e località, modificabili) e l'elenco delle attività, ciascuna con orario, luogo, categoria, costo e stato. Sono disponibili l'aggiunta, la modifica e l'eliminazione di un'attività e il cambio ciclico del suo stato (da fare → fatta → annullata) con un semplice tocco sul relativo badge.
- **Checklist** — in alto una barra di avanzamento riassume quanta parte della lista è già stata completata, considerando tutte le voci del viaggio e non solo quelle mostrate dal filtro attivo. Segue il pulsante di generazione della packing list in base alla tipologia del viaggio, poi i filtri (Tutti / Da completare / Completati) e infine le voci, raggruppate per categoria: ogni gruppo riporta il proprio conteggio "completati/totali" e dispone di un campo per aggiungere rapidamente nuove voci in coda. Ogni elemento è spuntabile con un tocco ed eliminabile singolarmente.
- **Spese** — un riquadro riepilogativo del budget in evidenza riporta la spesa effettiva e, a seconda del segno, il residuo o lo sfioramento rispetto al budget; una barra impilata distingue visivamente la quota già spesa da quella soltanto prevista. Al di sotto, una barra per ogni categoria ne mostra il peso relativo sul totale, un filtro consente di isolare una singola categoria e l'elenco dei movimenti mostra ciascuna spesa con importo e badge *Effettiva/Prevista*. Da qui si aggiungono o si modificano le spese tramite il relativo modulo.

- **Info** — un riquadro riepilogativo raccoglie i dati principali del viaggio (destinazione, partecipanti, budget) e sotto di esso si trova l’elenco delle note, ciascuna classificata per tipo (prenotazione, promemoria, indirizzo, altro) con un badge colorato. Le note si creano e si modificano toccandole, e si eliminano con pressione prolungata previa conferma.

In fondo alla schermata, un pulsante consente di eliminare il viaggio; poiché si tratta di un’azione distruttiva e irreversibile, è protetta da una richiesta di conferma e innesca la cancellazione a cascata di tutte le entità collegate (Sezione 5.5).

3.3.3 Timeline

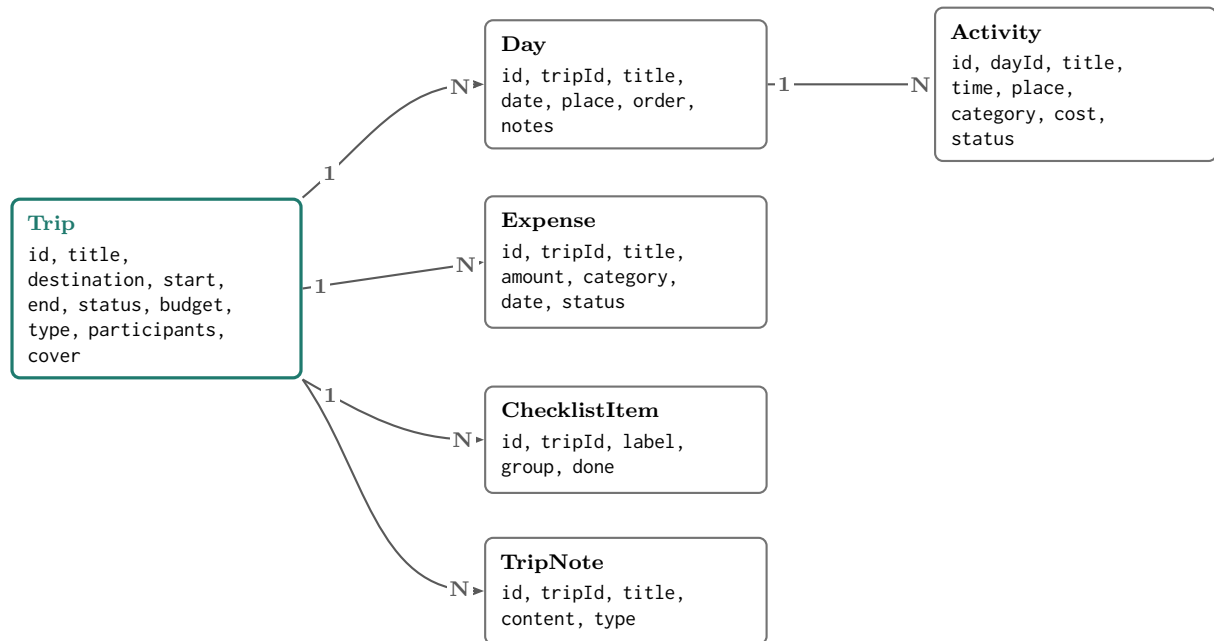
Offre una lettura d’insieme dell’itinerario come linea del tempo verticale: ogni giornata è segnata da un “pallino” collegato da una linea continua al giorno successivo, in modo da restituire visivamente la sequenza e la continuità del viaggio. Per ciascuna giornata sono riportati l’etichetta di data e località e, al di sotto, le attività del giorno in sola lettura, presentate con lo stesso aspetto grafico della scheda Itinerario per garantire continuità visiva tra le due viste. La Timeline è pensata per la consultazione — utile a colpo d’occhio prima o durante il viaggio — e per questo non espone comandi di modifica, che restano concentrati nella scheda Itinerario; vi si accede dal banner del dettaglio, che le trasmette l’identificatore del viaggio.

3.3.4 Statistiche

È la seconda sezione della barra inferiore e offre una lettura d’insieme che riunisce i dati di tutti i viaggi. La schermata raccoglie: un blocco di contatori (viaggi totali, in programma, in corso, completati); un grafico a torta della distribuzione dei viaggi per stato, con relativa legenda; un riepilogo economico che confronta la spesa effettiva con quella prevista — tenendo conto, tra le previsioni, anche dei costi delle attività non annullate — e con il budget complessivo, evidenziando il residuo o l’eventuale sforamento; lo stato di completamento complessivo delle checklist, calcolato su tutte le voci di tutti i viaggi; un grafico a barre delle spese per categoria; e la classifica dei viaggi con più attività pianificate, con il relativo rapporto di completamento. Poiché questi dati dipendono da informazioni modificabili nelle altre schermate, la sezione si aggiorna automaticamente ogni volta che viene riaperta, così da riflettere sempre lo stato più recente anche dopo modifiche effettuate altrove nell’applicazione.

3.4 Organizzazione dei dati

Il dominio è modellato con **sei entità** collegate da relazioni uno-a-molti che ruotano attorno al viaggio. Ogni entità è descritta da un tipo TypeScript esplicito (si veda la Sezione 5.2); i riferimenti tra entità sono realizzati con campi identificatore (`tripId`, `dayId`) che fungono da chiavi esterne a livello applicativo.



In sintesi: un **Trip** possiede molte **Day**, molte **Expense**, molte **ChecklistItem** e molte **TripNote**; ogni **Day** possiede molte **Activity**. Le attività non fanno riferimento diretto al viaggio ma alla giornata, cosicché l'itinerario risulta gerarchico (viaggio → giornata → attività). Questa scelta rende naturale sia la vista giorno-per-giorno sia la timeline, e semplifica la cancellazione a cascata (Sezione 5.5).

4 Scelte tecnologiche

4.1 Framework utilizzato

L'applicazione è sviluppata in **React Native** (versione 0.81) con il toolkit **Expo** (SDK 54) e scritta interamente in **TypeScript** (5.9) in modalità **strict**. La scelta di React Native rispetto a Flutter — i due principali framework per lo sviluppo mobile multiplatforma — risponde a tre esigenze: una singola codebase eseguibile su iOS e Android, la familiarità con l'ecosistema JavaScript/TypeScript e la ricchezza di librerie mature per navigazione, grafici e componenti nativi. Expo, in particolare, semplifica l'avvio del progetto, la gestione delle dipendenze native e il test su dispositivo reale tramite l'app **Expo Go**, accorciando sensibilmente il ciclo di sviluppo.

L'adozione di TypeScript in modalità **strict** porta type-safety sull'intero modello dei dati: ogni entità, ogni store e ogni proprietà di componente è tipizzata, con la conseguenza che molti errori (campi mancanti, stati non validi, riferimenti errati) vengono intercettati già in fase di compilazione anziché a runtime.

4.2 Librerie principali

Libreria	Ruolo nel progetto
React Native + Expo	Framework di base e runtime multiplatforma.
TypeScript	Tipizzazione statica dell'intero progetto.
Zustand	Gestione dello stato globale reattivo (uno store per dominio).
AsyncStorage	Persistenza locale chiave-valore su dispositivo.
React Navigation	Navigazione: native-stack + bottom-tabs.
react-native-gifted-charts	Grafici a torta e a barre nella schermata Statistiche.
react-native-svg	Rendering del gradiente vettoriale del banner del viaggio; è inoltre la dipendenza su cui si appoggiano i grafici.
@react-native-community/- datetimepicker	Selettori nativi di data e ora nei moduli.
expo-linear-gradient	Dichiarata come dipendenza ma non utilizzata: il gradiente del banner è realizzato direttamente con react-native-svg.
safe-area-context / screens	Gestione delle aree sicure e ottimizzazione delle transizioni.

4.3 Modalità di gestione della persistenza

Il salvataggio dei dati si appoggia ad AsyncStorage attraverso due sole funzioni di supporto, valide per qualunque tipo di dato:

- `loadJSON<T>(key, fallback)` — legge il contenuto associato a una chiave e lo converte dal formato JSON; se la chiave non esiste o si verifica un errore, restituisce un valore predefinito;
- `saveJSON<T>(key, value)` — converte il valore in formato JSON e lo salva sul dispositivo.

Ogni ambito dell'applicazione conserva i propri dati sotto una chiave dedicata, riconoscibile dal prefisso `to:` (`to:trips`, `to:days`, `to:activities`, `to:expenses`, `to:checklist`, `to:notes`). Ad ogni modifica lo stato viene prima aggiornato in memoria, così che l'interfaccia si aggiorni immediatamente, e subito dopo salvato su disco: in questo modo ciò che l'utente vede e ciò che è memorizzato restano sempre allineati. La coerenza tra i diversi insiemi di dati è infine assicurata dal meccanismo di eliminazione a cascata descritto nella Sezione 5.5.

5 Implementazione

5.1 Organizzazione del codice

Il codice sorgente è raccolto nella cartella `src/` e suddiviso per responsabilità, così che ogni tipo di file abbia una collocazione prevedibile. Questa separazione (modello, stato, persistenza, presentazione, moduli, utilità) è alla base della manutenibilità dichiarata tra i requisiti non funzionali.

```
src/
  types/           - definizioni dei tipi (Trip, Day, Activity, ...)
  stores/          - store Zustand, uno per dominio
  storage/         - livello di persistenza e cleanup a cascata
  screens/         - schermate; tabs/ per le schede interne
  forms/          - moduli modali di inserimento/modifica
  components/     - componenti UI riutilizzabili
  features/        - servizi di dominio (duplicazione viaggio)
  constants/       - categorie, etichette, template packing
  navigation/     - configurazione dei navigator
  utils/          - funzioni di supporto (id, date, colori, stato)
  theme/          - palette e token di stile
```

5.2 Modello dei dati

Ogni entità è un tipo TypeScript esplicito. Gli stati sono modellati con unioni di stringhe letterali, che rendono impossibili valori fuori dominio già in compilazione. Di seguito i tre tipi centrali dell'itinerario e del viaggio.

```
export type TripStatus = 'upcoming' | 'ongoing' | 'completed';

export type Trip = {
  id: string;
  title: string;
  destination: string;
  start: string;           // data ISO
  end: string;             // data ISO
  status: TripStatus;
  budget: number;
  type: string;            // tipologia: mare, montagna, città, ...
  participants: string[];
  cover: [string, string]; // 2 colori per il gradiente della copertina
};

export type ActivityStatus = 'todo' | 'done' | 'cancelled';

export type Activity = {
  id: string; dayId: string; title: string; time: string; // "HH:mm"
  place: string; category: string; cost: number; status: ActivityStatus;
};
```

5.3 Gestione dello stato

Lo stato globale è distribuito su **cinque store Zustand**, uno per dominio: quello dei viaggi, dell'itinerario, delle spese, delle checklist e delle note. Ogni store espone lo stato, un metodo `load()` che idrata i dati dal disco all'avvio, dei selettori (funzioni pure che filtrano/derivano i dati per la UI) e le operazioni di mutazione. Il pattern ricorrente è: muta in memoria, notifica la UI, scrivi su disco. Lo store dei viaggi ne è l'esempio più rappresentativo:

```
export const useTripStore = create<TripState>((set, get) => ({
  trips: [], search: '', statusFilter: 'all',

  load: async () => {
    let data = await loadJSON<Trip[]>(KEY, []);
    if (data.length === 0) data = seed; // dati di esempio al 1o avvio
    // ricalcola lo stato di ogni viaggio in base alla data odierna
    const updated = data.map((t) => ({ ...t, status: computeTripStatus(t.start, t.end) }));
    ;
    await saveJSON(KEY, updated);
    set({ trips: updated });
  },

  filtered: () => {
    const { trips, search, statusFilter } = get();
    return trips.filter((t) => {
      const okStatus = statusFilter === 'all' || t.status === statusFilter;
      const okSearch = (t.title + ' ' + t.destination)
        .toLowerCase().includes(search.toLowerCase());
      return okStatus && okSearch;
    });
  }
});
```

```

    },

    add: async (t) => {
      const withStatus = { ...t, status: computeTripStatus(t.start, t.end) };
      const next = [...get().trips, withStatus];
      set({ trips: next }); await saveJSON(KEY, next);
    },
    // update / deleteTrip seguono lo stesso schema
  ));

```

I componenti si agganciano allo store tramite l'hook `useTripStore(...)`, opzionalmente con un selettore (ad es. `useTripStore((s) => s.getById(id))`) per ri-renderizzare solo quando cambia il dato osservato. La combinazione di stato in memoria (reattivo) e persistenza asincrona realizza, di fatto, un pattern observer: ogni scrittura nello store propaga automaticamente l'aggiornamento a tutte le viste iscritte.

5.4 Gestione della navigazione

La navigazione è configurata in `RootNavigator`. Un `Tab Navigator` incapsula le due sezioni; la sezione Viaggi è uno `Stack Navigator` tipizzato tramite un tipo dei parametri di rotta, che documenta e vincola i dati passati tra schermate.

```

export type TripsStackParams = {
  TripsList: undefined;
  TripDetail: { tripId: string };
  Timeline: { tripId: string };
};

function TripsStack() {
  return (
    <Stack.Navigator>
      <Stack.Screen name="TripsList" component={TripsListScreen} options={{ title: 'I tuoi viaggi' }} />
      <Stack.Screen name="TripDetail" component={TripDetailScreen} options={{ headerShown: false }} />
      <Stack.Screen name="Timeline" component={TimelineScreen} options={{ headerShown: false }} />
    </Stack.Navigator>
  );
}

```

La schermata di dettaglio recupera il `tripId` dai parametri di rotta (`useRoute`) e naviga verso la `Timeline` propagandolo (`navigate('Timeline', { tripId })`). Le quattro schede interne, invece, sono governate da uno stato locale (`useState`) e ricevono il `tripId` come proprietà: questo evita di moltiplicare le rotte per contenuti che appartengono logicamente alla stessa schermata.

5.5 Gestione dei dati persistenti e integrità referenziale

Il livello di persistenza (`storage/storage.ts`) è volutamente minimale: due funzioni generiche `loadJSON`/`saveJSON` racchiudono `AsyncStorage` con gestione degli errori. Il punto architetturalmente più interessante è come viene garantita la **coerenza referenziale** in assenza di un database relazionale che offra vincoli e `ON DELETE CASCADE`.

La soluzione adottata è un **registro di hook di cleanup**, definito nel modulo `storage/-cleanup.ts`. All'importazione, ogni store che possiede dati dipendenti dal viaggio registra una propria funzione di pulizia. Quando un viaggio viene eliminato, lo store dei viaggi esegue tutti gli hook registrati prima di rimuovere il viaggio, ottenendo così una cancellazione a cascata applicativa che non lascia record orfani.

```
// storage/cleanup.ts -- registro degli hook
const hooks: CleanupFn[] = [];
export function registerTripCleanup(fn: CleanupFn): void { hooks.push(fn); }
export async function runTripCleanup(tripId: string): Promise<void> {
  for (const fn of hooks) await fn(tripId);
}

// tripStore.ts -- la cancellazione innesca la cascata
deleteTrip: async (id) => {
  await runTripCleanup(id); // rimuove giorni, attivita, checklist, spese, note
  const next = get().trips.filter((t) => t.id !== id);
  set({ trips: next }); await saveJSON(KEY, next);
},

// itineraryStore.ts -- hook che elimina giorni e relative attivita
registerTripCleanup(async (tripId) => {
  const st = useItineraryStore.getState();
  const dayIds = st.days.filter((d) => d.tripId === tripId).map((d) => d.id);
  const days = st.days.filter((d) => d.tripId !== tripId);
  const activities = st.activities.filter((a) => !dayIds.includes(a.dayId));
  useItineraryStore.setState({ days, activities });
  await saveJSON(DAYS, days); await saveJSON(ACTS, activities);
});
```

5.6 Componenti e widget principali

La presentazione è costruita con componenti riutilizzabili e con moduli modali per l'inserimento dei dati:

- TripCard — scheda di anteprima del viaggio nell'elenco (copertina, titolo, date, stato).
- ActivityCard — riga attività con indicatore di categoria, orario, luogo, costo e badge di stato commutabile.
- DayChip — “pillola” di selezione della giornata nell'itinerario (giorno della settimana + numero).
- TabIcons — icone della barra inferiore realizzate come piccoli componenti vettoriali.
- TripForm, DayForm, ActivityForm, ExpenseForm, NoteForm — moduli modali che gestiscono sia la creazione sia la modifica (precompilando i campi quando ricevono un elemento esistente) e applicano una validazione minima prima del salvataggio.

5.7 Approfondimenti implementativi

Si descrivono di seguito quattro meccanismi implementativi di particolare interesse tecnico, utili a chiarire come sono state realizzate alcune delle funzioni presentate in precedenza.

5.7.1 Calcolo automatico dello stato del viaggio

Lo stato di un viaggio non è memorizzato in modo statico ma derivato dalle date confrontate con la data odierna, e ricalcolato ad ogni caricamento e ad ogni salvataggio. Così l'elenco mostra sempre lo stato corretto senza intervento dell'utente.

```
export function computeTripStatus(start: string, end: string): TripStatus {
  const today = new Date(); const s = new Date(start); const e = new Date(end);
  today.setHours(0,0,0,0); s.setHours(0,0,0,0); e.setHours(0,0,0,0);
  if (today < s) return 'upcoming';
  if (today > e) return 'completed';
  return 'ongoing';
}
```

```
}

```

5.7.2 Generazione automatica delle giornate

Alla prima apertura dell'itinerario, l'applicazione genera automaticamente una giornata per ciascuna data compresa tra la partenza e il ritorno. La generazione avviene una sola volta: prima di procedere, l'applicazione verifica se per quel viaggio esistono già delle giornate e, in tal caso, non ne crea di nuove. Si evita così che le giornate vengano duplicate ad ogni successiva apertura della scheda.

```
ensureDaysForTrip: async (tripId, start, end) => {
  const already = get().days.some((d) => d.tripId === tripId);
  if (already || !start || !end) return; // idempotente
  const generated: Day[] = []; let order = 1;
  for (let d = new Date(start); d <= new Date(end); d.setDate(d.getDate() + 1)) {
    generated.push({ id: makeId(), tripId, title: '', date: d.toISOString(),
      place: '', order: order++, notes: '' });
  }
  const next = [...get().days, ...generated];
  set({ days: next }); await saveJSON(DAYS, next);
},
```

5.7.3 Duplicazione di un viaggio

La duplicazione è utile a chi viaggia spesso, perché molti viaggi si assomigliano: stesse tappe, itinerari simili, una checklist di preparazione ormai roduta e categorie di spesa ricorrenti. Ricrearli ogni volta da zero è ripetitivo e ci si dimentica facilmente qualcosa. La funzione permette invece di partire da un viaggio già esistente e ottenerne una copia completa - con giornate, attività, checklist, spese e note - su cui basta aggiornare le date e i dettagli propri della nuova occasione. Un viaggio già fatto, o uno preparato con cura, diventa così un modello riutilizzabile che riduce sensibilmente il lavoro di inserimento.

Il servizio `duplicateTrip` copia il viaggio con tutti i dati collegati. L'aspetto più delicato è assegnare a ogni elemento copiato un nuovo identificatore senza però perdere i collegamenti tra i dati: poiché ogni attività è legata a una giornata, si tiene traccia della corrispondenza tra il vecchio e il nuovo identificatore di ciascuna giornata, così da riagganciare ogni attività alla giornata copiata corretta. Gli stati vengono inoltre riportati alla condizione iniziale, perché la copia rappresenta un nuovo viaggio ancora da svolgere.

```
export async function duplicateTrip(tripId: string): Promise<string> {
  const original = trips.getById(tripId);
  const newTripId = makeId();
  await trips.add({ ...original, id: newTripId,
    title: `${original.title} (copia)`, status: 'upcoming' });

  const dayIdMap = new Map<string, string>(); // vecchio -> nuovo id
  for (const d of itin.daysOf(tripId)) {
    const newDayId = makeId(); dayIdMap.set(d.id, newDayId);
    await itin.addDay({ ...d, id: newDayId, tripId: newTripId });
  }
  for (const [oldDayId, newDayId] of dayIdMap) // attività rimappate
    for (const a of itin.activitiesOf(oldDayId))
      await itin.addActivity({ ...a, id: makeId(), dayId: newDayId, status: 'todo' });
}
```

```

    for (const i of checklist.itemsOf(tripId)) await checklist.add(newTripId, i.label, i.group);
    for (const e of expenses.expensesOf(tripId)) await expenses.add({ ...e, id: makeId(), tripId: newTripId, status: 'planned' });
    for (const n of notes.notesOf(tripId)) await notes.add({ ...n, id: makeId(), tripId: newTripId });
    return newTripId;
}

```

5.7.4 Packing list generata dal tipo di viaggio

La generazione unisce un blocco base comune con il blocco specifico della tipologia del viaggio (es. mare, montagna), quindi aggiunge solo le voci non ancora presenti nella checklist, confrontando le etichette in modo case-insensitive per non introdurre duplicati. La funzione restituisce il numero di voci effettivamente aggiunte, così che l'interfaccia possa dare un riscontro all'utente.

```

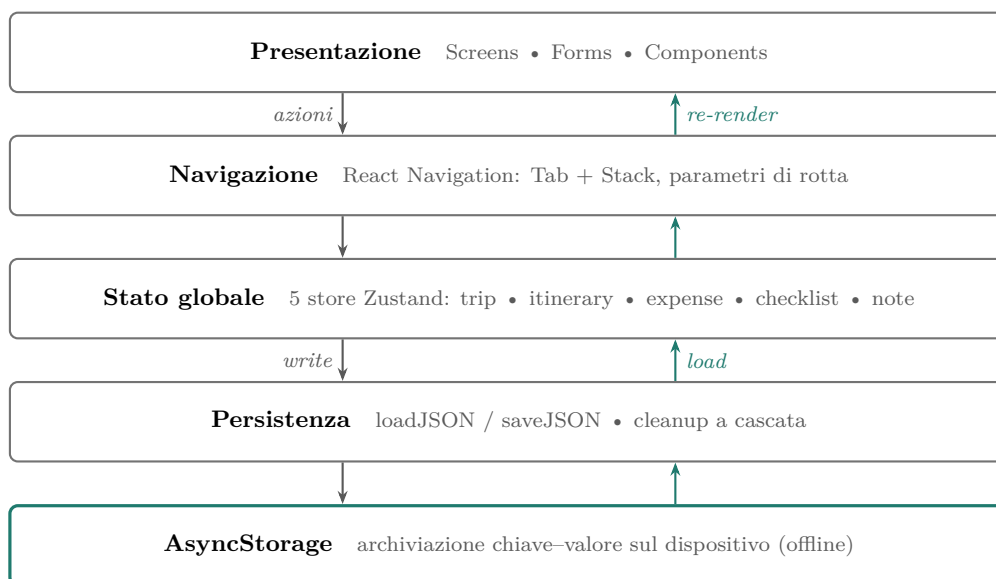
generatePacking: async (tripId, type) => {
    const merged: Record<string, string[]> = {};
    for (const block of [packingTemplates.base, packingTemplates[type] ?? {}])
        for (const [g, arr] of Object.entries(block))
            merged[g] = [...(merged[g] ?? []), ...arr];

    const existing = new Set(get().items.filter((i) => i.tripId === tripId)
        .map((i) => i.label.toLowerCase()));
    const toAdd: ChecklistItem[] = [];
    for (const [group, arr] of Object.entries(merged))
        for (const label of arr)
            if (!existing.has(label.toLowerCase())) {
                toAdd.push({ id: makeId(), tripId, label, group, done: false });
                existing.add(label.toLowerCase());
            }
    if (toAdd.length) { const n = [...get().items, ...toAdd]; set({ items: n }); await saveJSON(KEY, n); }
    return toAdd.length;
},

```

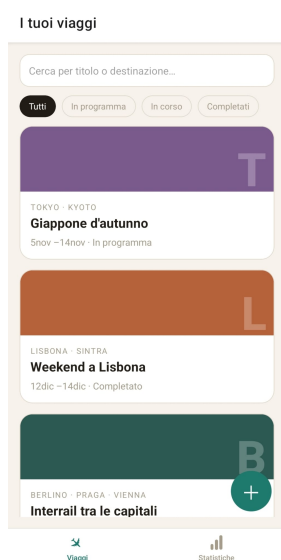
5.8 Architettura a livelli in sintesi

Le scelte descritte compongono un'architettura a livelli con un flusso di dati unidirezionale e ben tracciabile: l'interazione dell'utente attraversa presentazione, navigazione e stato, fino alla persistenza; le modifiche allo stato risalgono automaticamente fino alle viste iscritte.

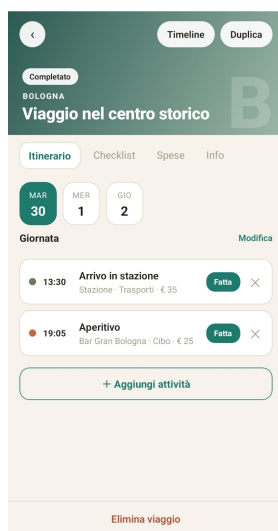


6 Schermate dell'applicazione

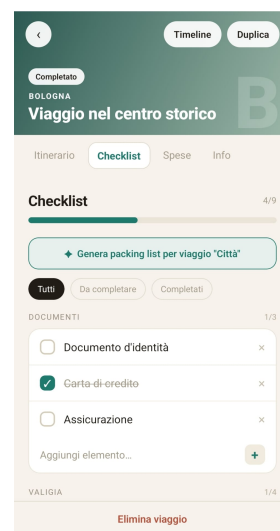
Questa sezione raccoglie gli screenshot delle principali schermate dell'applicazione, così da affiancare alla descrizione testuale un riscontro visivo dell'interfaccia effettivamente realizzata.



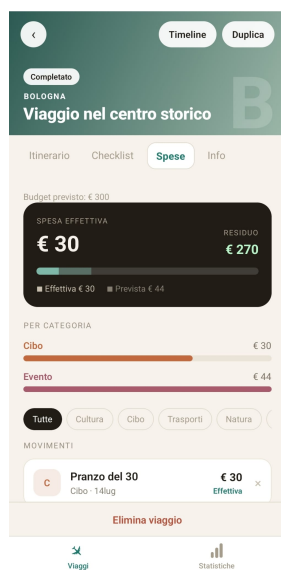
Elenco viaggi



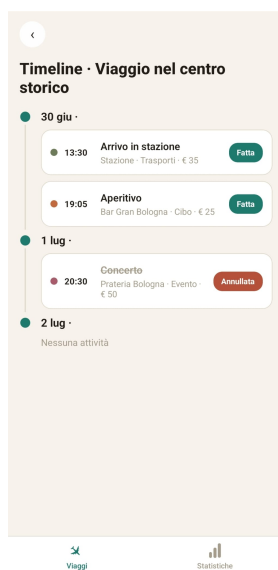
Dettaglio • Itinerario



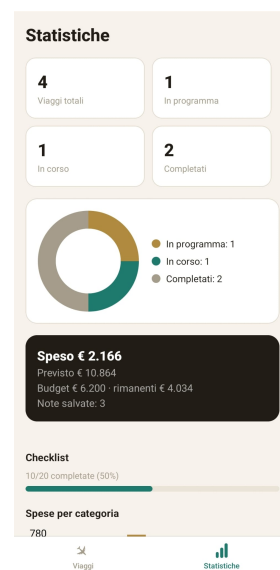
Dettaglio • Checklist



Dettaglio • Spese



Timeline



Statistiche



Statistiche

7 Conclusioni

TripMate è un'applicazione mobile completa per l'organizzazione dei viaggi, che copre l'intero ciclo — pianificazione dell'itinerario, preparazione, gestione delle spese e resoconto finale — attraverso la gestione integrata di viaggi, giornate, attività, checklist, spese e note. A queste funzioni di base l'applicazione affianca ricerca e filtri, una sezione di statistiche aggregate e tre funzionalità aggiuntive — la timeline giornaliera dell'itinerario, la duplicazione di un viaggio e la generazione della checklist di preparazione in base alla tipologia — inserite organicamente nel flusso d'uso. L'intero sistema opera senza connettività e conserva i dati tra sessioni successive grazie alla persistenza locale sul dispositivo.

Sul piano tecnico, l'applicazione è costruita seguendo un'organizzazione a livelli: l'interfaccia è separata dalla logica di gestione dei dati, che a sua volta è suddivisa per ambito (viaggi, itinerario, spese, checklist, note) e appoggiata a un'archiviazione locale sul dispositivo. Le informazioni restano sempre allineate tra ciò che l'utente vede e ciò che è salvato, e l'eliminazione di un viaggio rimuove automaticamente tutti i dati collegati, così da non lasciare informazioni incoerenti anche senza un database tradizionale. L'uso di TypeScript e la suddivisione del codice per ambiti rendono infine il progetto ordinato e più semplice da mantenere e da estendere.